Job Submission Guidelines: UMICH – Life Sciences Institute
jporta@umich.edu

This document contains information on the computing resources available to Ohi lab members and some basic guidelines on submitting jobs to the high-performance computing (HPC) cluster, mainly the *login1* and *login2* nodes. The LSI intranet also has a useful section on job submission and provides additional information on the available hardware. Some of the information for this document was taken from the LSI intranet and the current Ohi Lab Computation Guide document in the lab Google Drive folder.

Tips:

1. **Memory issues:** Users who are running large jobs (i.e. many images, large particle boxes) often experience memory issues. When it's an MPI (message passing interface) job the error output is complicated and unclear as to the source of the problem. There are two options for minimizing memory issues:
    - Allocate more memory when using the PBS scheduling system. This can be done by modifying your qsub.sh script line: #PBS -l nodes=1:ppn=2,pmem=12g
    - Use the large memory node (himem#, see below).
    - Note: do not allocate more than 12 GB of memory as some must be leftover for basic job handling, which means that your job will likely crash.

2. **Multithreading on *mohi-gpu02.lsi.umich.edu*:** Each of the 4 GPU nodes can run multiple threads. In the past some users assumed that if each node is occupied (as determined from the 'nvidia-smi' command) that they cannot run jobs. This is not true as each node can run multiple jobs via multithreading.

3. **MPI errors:** As stated above, MPI errors are esoteric and confusing making it difficult to troubleshoot your job. Despite this, many MPI errors are related to erroneous user input and can be overcome by carefully reviewing your job input parameters.

Terminology:

- *Cluster* - a group of inter-connected computers that work together to perform computationally intensive tasks. In a cluster, each computer is referred to as a "node".
- *Node* - any physical device within a network of other tools that is able to send, receive, or forward information. A personal computer is the most common node. It's called the computer node or internet node.
- *Head node* - the computer to which you log in, and where you edit scripts, compile code, and submit jobs.
- *Compute node* - the nodes in which a job is submitted to using a scheduling system (i.e. PBS). Each node contains one or more processors (CPUs or GPUs; usually two or more)

on which computation takes place. For example, the *login1* node has two physical processors, each of which has 12 cores. This means that the login1 node can perform 24 tasks simultaneously.

- *Core* - the main (or 'brain') functional unit(s) of a physical where all calculations take place.
- *Thread count* - the number of individual processes that can run simultaneously on a CPU.

Available Hardware:

1. HPC Cluster:
    - Login nodes (2) | login.hpc.lsi.umich.edu
    - *login1.hpc.lsi.umich.edu*
        - 2 x Intel Xeon E5-2680 2.5 GHz processors
        - 12 cores per physical processor
        - 24 cores total
        - 256 GB memory

    - *login2.hpc.lsi.umich.edu*
        - 2 x Intel Xeon CPU E5-2660 2.60GHz processors
        - 20 Cores per physical processor
        - 40 Cores total
        - 256 GB memory

    - Large memory node (1) | himem-##
        - 4 x Intel Xeon E7-8857 v2 3.00 GHz Processors
        - 12 Cores per physical processor
        - 48 Cores total
        - 512 GB memory

    - GPU nodes (6) | gpucomp-##
        - 2 x Intel Xeon Gold 6128 3.4 GHz Processors
        - 6 Cores per physical processor
        - 12 Cores total
        - 384GB memory
        - 4 x NVIDIA GeForce RTX 2080 Ti GPUs
        - Each has 11 GB memory

2. Ohi lab (*mohi-gpu02.lsi.umich.edu)* GPU nodes (4):
    - NVIDIA GeForce GTX 1080 Ti (GPU 0,1,2,3) GPU cards
        - 3584NVIDIA CUDA® Cores 1582Boost Clock (MHz)
        - Each has 11 GB of memory

3. Ohi lab (*mohi-gpu01.lsi.umich.edu)* GPU nodes (2):
    - NVIDIA GeForce GTX Titan XP (GPU 0,1) GPU cards
        - 2688CUDA Cores 837base and boost clock (MHz)
        - Each has 12 GB of memory
        - This system is designated for running cryoSPARC jobs

4. Linux workstations:
    - There are slight variations in the hardware, so this example is for the *mdo7.lsi.umich.edu* workstation.
        - 1 x Intel Xeon E3-1230 3200 MHz (1 core)
        - 1 x NVIDIA GP107GL Quadro P600 GPU card
        - 16 MB of memory
        - example: PHENIX real-space refinement of a 4.0 Å map with structure validation and simulated annealing took 23 minutes

5. iMac Desktops (for those who have them):
    - This example is with the desktop at my desk in the computer room:
        - 1 x Intel 3.5 GHz Core i5 processor
        - 32 GB 2400 MHz DDR4 RAM
        - ATI Radeon Pro 575 4 GB GPU card
        - example: the same PHENIX real-space refinement job as above took 22 minutes
6. Personal laptops: Modern laptops are powerful and almost certainly having multicore processors, GPU cards and approximately 12 GB of memory or more. If your job is small this is a good option that could lighten the load on the shared computing resources. If you need assistance setting up your laptop/shell environment, feel free to ask.

Putting the 'Shared' in Shared Computing (HPC etiquette):

1. **Don't run jobs on the head node:** This node is responsible for processing logins, job scheduling, transferring files in and out, and compiling code. Running jobs on the head node will slow down the system for everyone else.
2. **Test large jobs first:** before submitting jobs to the queuing system. Before you submit any large number or long running jobs, submit a test run using 1-2 cores. Once you have determined that these jobs are working successfully, scale your problem up. For example, to first test if your job parameters are correct, change the number of nodes and processes to 1 (#PBS -l nodes=1:ppn2).
3. **Monitor your jobs periodically:** jobs crash all the time and if not monitored you can waste a lot of time while needlessly occupying nodes. A quick 'qstat -a' command every so often will save you a lot of time.
4. **Don't 'hog' up the CPUs!** There are approximately 1,200 simultaneous tasks that can be run on the cluster. If your qsub script requests 'nodes=10:ppn=20' that is 200 tasks, and therefore ~1/6th of the cluster resources. Be mindful of others who also need to complete

jobs. If you need to run a job that requires a lot of resources, try to avoid queuing multiple jobs at once.

5. **Which system should you run your job on?** There are no simple and straightforward rules on this, so each user will need to determine this for each job. However, there are a few considerations that apply to every job:
    - CPU vs GPU: This will be an easy decision if the job you're running isn't supported for GPU processing (i.e. certain RELION jobs). For particles with large boxes (> 700 $pixel^2$) you will need to run your job on the CPU cluster. If you know that your job is going to take a long time it is best to submit it to the cluster as jobs running interactively from the terminal are more prone to unexpectedly aborting.
6. **Data binning:** This will be job and data specific, but if you can bin your data (i.e. particle images) this will significantly speed-up computation and free up more resources for other users.

PBS Batch Submission (command line):

1. Scripts for job submission (command line): These can be found in the lab directory of the filesystem (/lsi/groups/mohilab/scripts). The file *qsub.sh* is a general template for job submission and contains useful comments.
2. Node allocation (nodes vs ppn): The most efficient use of the computing resources is to request more cores (i.e. processes per node) than nodes. For example, if you request 10 nodes and 20 processes per node (i.e. #PBS -l nodes=10:ppn=20) you will spread your job across 200 tasks and not needlessly occupy an entire node while using only a fraction of the cores for that node (i.e. #PBS -l nodes=10:ppn=2).

RELION 3.0.8 (and beyond…):

- The newer versions of RELION have a revamped user interface with additional options for job submission. As of right now (02/13/2020) version 3.1 is still rather new and users have been experiencing job errors. Therefore, if you do not need a feature specific to version 3.1 it is recommended that you use version 3.0.8. Some important things to consider are:

    - **Version(s) and whether to use a module version or the SBGrid version:** This is going to depend on which system you are running your job on. If you are submitting a CPU job to the cluster, module version 'relion/3.0.8-cluster/openmpi/4.0.2' is stable and widely used. For a GPU job submission module version 'relion/3.0.8-cluster/gpu/openmpi/4.0.2' is recommended. For GPU jobs on *mohi-gpu02.lsi.umich.edu* the SBGrid version is stable.
    - **Refine3D FSC calculation:** Relion carries out the gold-standard FSC calculation in a way that requires the user to request an odd number of nodes. This was done intentionally by the developers due to a bug in recent version that caused some of the particles in the first half-set to be ignored when the user requested an even number of nodes. When not done properly the user will get a confusing MPI error.

For Refine3D jobs make sure to request an odd number of nodes to avoid your job from aborting.

- **The 'Compute' tab (user interface):** The main consideration here is to *not* turn on 'parallel disk I/O' and 'pre-read all particles into RAM'. If you are using GPU acceleration, you can leave the 'Which GPUs to use' field blank and the program will allocate as many nodes as possible.
- **The 'Running' tab:** The source of many headaches for new and experienced users alike. This is the part that has become more confusing to users with the version 3.x release. Here is an explanation of each field:

    - *Number of MPI procs*: This is the number of nodes you are requesting and applies to both CPU and GPU jobs. However, if you are running your job on the *mohi-gpu02.lsi.umich.edu* system from the command line, you only need to ensure that the value is greater than 1. For example, if the number of MPI procs is 1 then the software will run the non-MPI version of the program '*relion_refine*' as opposed to the MPI version '*relion_refine_mpi*'.
    - *Number of threads*: Multithreading will allow you to perform more tasks simultaneously. Take advantage of this feature as it will significantly speed-up computation. Requesting 16 threads is recommended though the users should be aware of memory limitations should they request too many threads.
    - *Submit to queue*: For a GPU job on *mohi-gpu02.lsi.umich.edu* keep this option turned off as the lab GPU system does not use a scheduling system (i.e. PBS). For CPU and GPU jobs being submitted to the cluster you will need to turn this on unless you plan to submit your job from the command line.
    - *Queue name*: For CPU jobs on the cluster the correct value is 'batch' (without the parenthesis) and 'gpu' for GPU jobs.
    - *Queue submit command*: The correct value here is 'qsub'.
    - *Wall time in hours for job*: This is important yet often overlooked. The wall time is set by the user for the job to abort. Setting a reasonable wall time will release the nodes that job is occupying should it not be complete by the time set. This is important in the context of jobs that have critical failures, yet do not cause the scheduling system to release the nodes.
    - *Number of nodes to request*: If you are submitting your job to the cluster, it is this value (and not the *number of MPI procs*) that is important. For jobs being submitted to the cluster these values are now passed to the qsub script (3.0.8.bash). In the past, these values were defined in the script by the user. However, if you are submitting your job from the command line ($ qsub qsub_batch.sh) you still need to define these in a text editor.
    - *Number of MPI processes to run per node*: The number of tasks (i.e. parallel jobs) is determined by this multiplied by the number of nodes. For example, if you request 2 nodes and 10 processes per node you will spread your job across 20 tasks. Recall that the best practice is to fewer nodes than processes.

- *Standard submission script*: Use the 3.0.8.bash script. This can also be found in the lab directory (/lsi/groups/mohilab/scripts).
- *Minimum number of dedicated cores per node*: This is useful parameter and will prevent you from not utilizing (or wasting) the maximum potential of each requested node. In general, a value of 20 is recommended.
- *Additional arguments*: This can be any job parameter not available in the user interface. For CPU jobs on the cluster make sure to use the '--cpu' option as this will force the software to use the CPU vector acceleration which will speed up computation. Note that certain jobs/options are not yet supported for vectorized computing (i.e. Refine3D with image alignment turned off) and will produce an error.

Miscellaneous items and future additions:

- It is possible that we can gain access to the university's Great Lakes Cluster. More details to come (or none) …
- Shell script to symbolically link micrographs on-the-fly to the user's lab directory as they are being collected on the microscope.
- Lab data storage practices and guidelines.
- Interactive batch and array jobs

Useful resources:

- LSI Intranet - Compute Cluster Guidlines and Job Submission - An LSI-specific resource with information on hardware, job submission and structural biology software.
- Ohio Supercomputer Center - An excellent general resource for all things computing cluster and batch submission.
- qsub manual page - Contains everything you wanted to know, including many things you don't about the qsub option flags. If you are working in a terminal you can load this with the 'man qsub' command.